

# LLM 后训练之 Distillation

知识 / 行为 / 推理 / 轨迹蒸馏：深度研究与工程手册

## 核心判断

Distillation 不是简单“让小模型模仿大模型答案”。它是把 Teacher 已经获得的概率结构、行为策略、推理轨迹或架构能力，在受限 Student 容量与部署预算下重新编码。真正的工程对象是 Transfer Contract：蒸馏什么、沿着什么分布蒸馏、用什么监督粒度、如何验证能力保真，以及是否值得支付 Teacher 生成与 Student 训练成本。

Research snapshot: 2015 - 2026 | 重点：Know-Why + Know-How

# 0. Executive Summary：蒸馏到底在做什么？

在经典知识蒸馏中，Teacher 的软概率分布提供了比 hard label 更丰富的“暗知识（dark knowledge）”。Hinton 等人 2015 年的框架强调使用 softened logits 将 ensemble 或大型模型的知识压入较小模型 [1]。对自回归语言模型，这个问题随后从 token 分布扩展到 sequence、rationale、reasoning trajectory，甚至 student 自己访问的 on-policy states。

因此，LLM Distillation 最好理解为一个受约束的信息传输问题：Teacher 有一个复杂策略  $\pi_T$ ，Student 只有有限容量  $\pi_S$ 。我们希望在部署约束下，最大化 Student 对“真正有价值的 Teacher 行为”的保留，而不是机械最小化某个 KL。

## 核心公式

Effective Distillation  $\approx$  Transferable Signal  $\times$  Student Relevance  $\times$  Coverage  $\times$  Verification  $\times$  Optimization Fit / (Teacher Cost + Student Cost + Capacity-Mismatch Risk)

| 问题             | 传统答案                | LLM 时代的修正                                                                          |
|----------------|---------------------|------------------------------------------------------------------------------------|
| 蒸馏什么？          | logits              | logits + sequence + rationale + preference + trajectory + hidden representation    |
| Teacher 越强越好吗？ | 通常默认是               | 不一定；teacher-student capacity gap、trace complexity、diversity 都可能使更强 teacher 更难学     |
| 数据从哪里来？        | 固定标注集               | teacher outputs、student on-policy outputs、verified search traces、real-user prompts |
| 怎么训练？          | forward KL          | SFT/SeqKD、forward/reverse KL、JSD、on-policy KD、多任务 rationale loss                   |
| 成功标准？          | 逼近 teacher accuracy | 能力保真 + 速度/显存/成本 + 分布外泛化 + 行为稳定性                                                    |

# 1. 为什么 Post-Training 需要 Distillation？

蒸馏在后训练流水线中的核心作用不是“替代 RL”，而是把昂贵模型或昂贵搜索过程已经发现的行为压入更便宜的 Student。DeepSeek-R1 的公开结果提供了一个非常清晰的案例：其研究报告指出，使用 DeepSeek-R1 生成的 reasoning data 直接 fine-tune Qwen/Llama dense base models，可以获得很强的推理能力；论文还明确报告，对 Qwen2.5-32B，直接从 R1 蒸馏优于在该小模型上直接做同类 RL [7]。

- 部署压缩：减少参数、显存、延迟和每次请求成本。
- 能力复制：把大模型已经学到的指令遵循、推理、工具策略迁移到小模型。
- 搜索摊销：Teacher/RL/Search 先付出昂贵探索成本，Student 通过 supervised / KD 学习“搜索后的结果”。
- 模型家族化：一个昂贵 Teacher 可以为多个不同尺寸、不同延迟目标的 Student 提供统一行为先验。
- 架构迁移：2026 年已有工作研究将 MoE Teacher 通过剪枝、专家组合与蒸馏转换为 dense Student [15]。

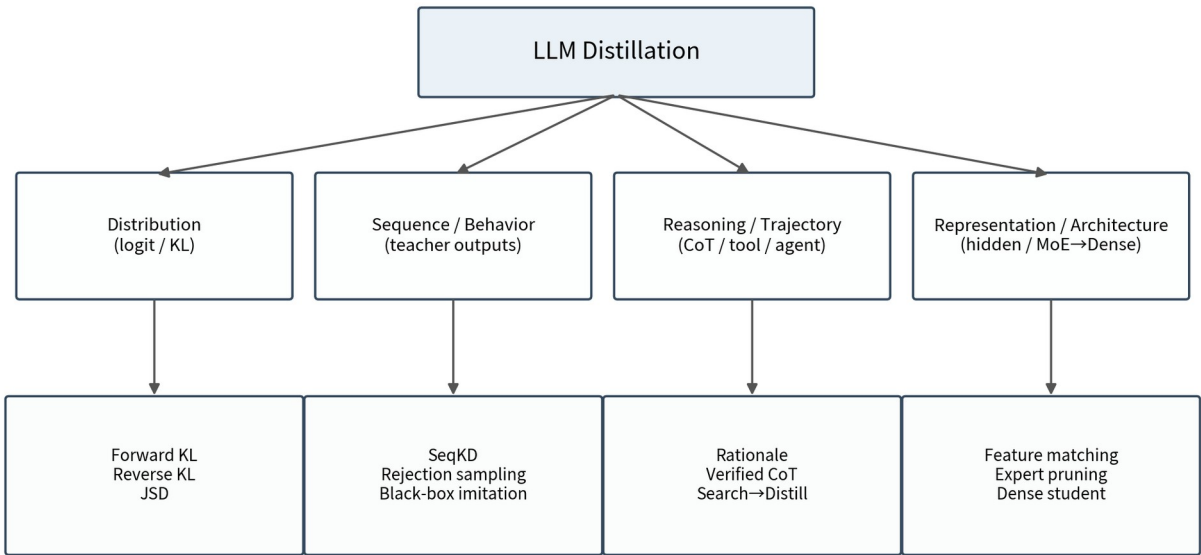
Reasoning Distillation : Search → Verify → Distill



关键：student 学到的是“teacher 已发现并验证的行为”，不必自己重新支付同等 RL / Search 成本

图 1：Reasoning Distillation 的典型工程闭环（本手册综合绘制）

2. Distillation Taxonomy：先分清“知识载体”



核心区别：蒸馏的“知识载体”不同 → 数据、损失、成本与失败模式也不同

图 2：LLM Distillation 的四类主要知识载体

| 类型                      | Teacher 暴露             | Student 学什么      | 典型方法                         | 适用场景                                |
|-------------------------|------------------------|------------------|------------------------------|-------------------------------------|
| Distribution / Logit KD | logits / probabilities | 下一 token 的完整概率结构 | soft-target KD, FKL/RKL/JSD  | white-box teacher, 精细概率匹配           |
| Sequence / Behavior KD  | 生成文本                   | 高质量行为序列          | SeqKD, teacher-generated SFT | black-box API、instruction following |

| 类型                               | Teacher 暴露                    | Student 学什么   | 典型方法                                | 适用场景             |
|----------------------------------|-------------------------------|---------------|-------------------------------------|------------------|
| Reasoning / Trajectory KD        | CoT / tool / multi-step trace | 策略、分解、反思、工具路径 | rationale SFT, verified CoT distill | 数学、代码、Agent、长程任务 |
| Representation / Architecture KD | hidden / expert activations   | 中间特征或结构知识     | feature matching, MoE→Dense         | 模型压缩、跨架构迁移       |

这四类并不是互斥的。成熟 recipe 往往同时使用 sequence data 先教会 student 行为，再用 token-level distribution matching 细化概率分布，并通过 on-policy data 修复 student 自身轨迹上的误差。

### 3. 经典 Logit Distillation：Soft Targets 为什么有信息？

经典 KD 通过 temperature T 对 Teacher logits 进行 softening，使非 top-1 类别之间的相对概率关系也成为训练信号 [1]。对于语言模型，可以把每个 token position 的 teacher distribution 看作局部策略。

**经典目标（简化）**

$$L = \alpha \cdot \text{CE}(y, p_S) + (1-\alpha) \cdot T^2 \cdot \text{KL}(p_{T \rightarrow T} \parallel p_{S \rightarrow T})$$
 Temperature 越高，分布越平滑；但过高会把容量有限的 Student 拉去覆盖 Teacher 的大量低概率区域。

#### 3.1 Forward KL vs Reverse KL

| 维度       | Forward KL D_KL(T    S)      | Reverse KL D_KL(S    T)           |
|----------|------------------------------|-----------------------------------|
| 直觉       | mode-covering                | mode-seeking                      |
| 惩罚重点     | Teacher 有概率、Student 漏掉的区域    | Student 放概率但 Teacher 不支持的区域       |
| 容量不足时风险  | Student 被迫覆盖过多 mode，可能平均化/扩散 | 可能丢失次要 mode，diversity 下降          |
| LLM 研究代表 | standard KD / GKD variants   | MiniLLM、部分 on-policy distillation |
| 工程建议     | 需要覆盖多样输出时更有意义                | 需要高精度、减少低概率胡言时常有优势                |

MiniLLM 报告用 reverse KL 替代传统 forward KL，并结合 on-policy optimization，在 instruction-following 任务上改善 precision、exposure bias、calibration 和长文本生成 [5]。GKD 则把 divergence 选择与“在哪些序列上蒸馏”解耦，允许 forward KL、reverse KL 或 generalized JSD [6]。

### 4. Sequence-Level Distillation：从概率模仿到行为模仿

Kim & Rush 2016 把 distillation 从 word-level prediction 扩展到 sequence level：Teacher 先产生完整输出，Student 再把这些 Teacher sequences 当训练目标。论文报告其最佳 Student 相比 Teacher 推理速度约快 10 倍且性能损失很小；结合 pruning 后参数可减少约 13 倍 [2]。

| 方案             | 训练数据                      | 优点              | 主要损失                         |
|----------------|---------------------------|-----------------|------------------------------|
| Hard-label SFT | human/ground-truth answer | 稳定、直接           | 没有 Teacher 分布信息              |
| SeqKD          | teacher best sequence     | 简单、black-box 可用 | Teacher diversity 被压成少量 mode |

| 方案                   | 训练数据                                   | 优点     | 主要损失      |
|----------------------|----------------------------------------|--------|-----------|
| Best-of-N distill    | teacher N candidates + verifier select | 质量高，可控 | 生成与验证成本高  |
| Multi-sample distill | teacher 多个有效 outputs                   | 保留多样行为 | 数据体积与冲突增加 |

### Know-Why

Sequence distillation 的本质是降低 Student 要拟合的输出熵：Teacher 的高质量 sequence 把“应该怎么答”的搜索空间先压缩了。但如果只保留单一答案，也会损失 Teacher 的多样性和不确定性。

## 5. Rationale / Reasoning Distillation：为什么“解释”能成为监督信号？

Distilling Step-by-Step 提出把 LLM rationale 作为额外 supervision，在多任务训练中同时学习 rationale 与 label。该工作报告，在其四个 NLP benchmark 上，这种 rationale supervision 能用更小模型和更少数据达到或超过传统 fine-tuning / distillation；其中一个实验中 770M T5 使用 80% 数据超过 few-shot 540B PaLM [3]。

Orca 进一步强调“浅层 imitation”会让 Student 学到 Teacher 的风格，却不一定学到 reasoning process；其方案使用 GPT-4 explanation traces、step-by-step thought processes 与 progressive learning 来训练 13B Student [4]。

| Reasoning supervision 粒度   | Student 获得什么 | 风险                              |
|----------------------------|--------------|---------------------------------|
| Answer only                | 最终映射         | credit signal 太稀疏，难学复杂分解        |
| Short rationale            | 中间因果/步骤      | 可能比长 CoT 更适合小 Student           |
| Long CoT                   | 反思、分支、验证等模式  | 冗余、错误中间态、长度成本、capacity mismatch |
| Process label / step score | 步骤质量信号       | 需要 PRM/verifier，构建成本更高          |
| Tool trajectory            | 何时调用工具、如何恢复  | 环境依赖、状态空间复杂                     |

2025 的 CoT distillation 实证研究进一步指出：更强 Teacher 并不总产生更强 Student；Teacher 的 trace diversity、复杂度与 Student 容量匹配同样重要。弱 Student 甚至可能更适合简化后的 CoT，而非最细粒度、最长的 reasoning trace [10]。

## 6. DeepSeek-R1：Search / RL → Distill 的代表案例

DeepSeek-R1 把 reasoning distillation 的工程逻辑展示得非常清楚：先让大模型通过 RL 发现反思、自验证、长 CoT 等高级行为，再利用其 reasoning data fine-tune 多个小型 dense base model [7]。这意味着 Student 不必重新支付等量 exploration cost，即可吸收 Teacher 已经发现的策略模式。

| 模型 / 结果             | 公开报告中的关键数字                                             |
|---------------------|--------------------------------------------------------|
| R1-Distill-Qwen-7B  | AIME 2024 55.5% [7]                                    |
| R1-Distill-Qwen-32B | AIME 2024 72.6%；MATH-500 94.3%；LiveCodeBench 57.2% [7] |

| 模型 / 结果    | 公开报告中的关键数字                                                         |
|------------|--------------------------------------------------------------------|
| Student 尺寸 | 开源 1.5B、7B、8B、14B、32B、70B distilled checkpoints [7]                |
| 方法结论       | Qwen2.5-32B 上，R1 reasoning data 的直接 distillation 优于在小模型上直接做 RL [7] |

### 重要限制

不能把“R1 distillation 成功”简单归因于长 CoT 越长越好。后续研究发现，不同模型家族对同一套 long-CoT traces 的可迁移性并不完全一致；trace segmentation、去冗余、修复中间错误等预处理可能显著影响 Student [11]。

## 7. Verified Distillation：为什么 Verifier 是 Reasoning 蒸馏的上限控制器？

当 Teacher 生成 reasoning traces 时，Teacher 本身也会犯错。如果把“Teacher confidence”直接当 correctness，错误会被批量复制。Reasoning distillation 因而越来越依赖 outcome verifier / unit tests / symbolic checks / reward model 做过滤。2025 的 1.4M distilled reasoning dataset 就明确对数学答案做 reference validation、代码用 tests 验证、其他任务用 reward model 辅助筛选 [12]。

| 阶段        | 推荐操作                               | 原因                           |
|-----------|------------------------------------|------------------------------|
| Generate  | 每题生成 N 个候选                         | 增加策略覆盖                       |
| Verify    | 优先使用 deterministic verifier        | 减少 Teacher hallucination 被复制 |
| Filter    | 去掉错误、重复、异常长/短 trace                | 提升 signal/noise              |
| Normalize | 保留关键步骤，减少无效自言自语                    | 匹配 Student 容量                |
| Train     | SFT / seq KD / token KD 混合         | 兼顾行为与分布                      |
| Eval      | 独立 benchmark + pass@k + token cost | 防止只学 “Teacher style”         |

## 8. On-Policy Distillation：从“Teacher 的状态”转到“Student 的状态”

### 为什么 On-policy Distillation 有意义？

固定 teacher data 只覆盖 teacher / 旧策略轨迹；student 部署时会访问自己的状态分布。

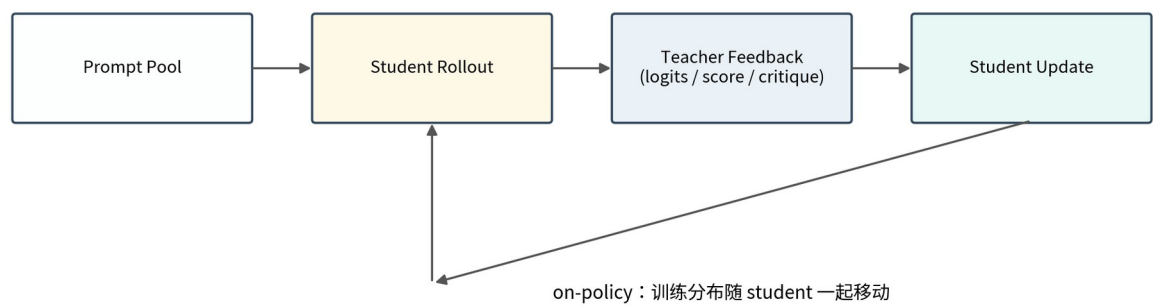


图 3：On-policy Distillation 的状态分布闭环

固定 Teacher-generated dataset 有一个根本问题：Student 训练时看到的是 Teacher 或旧策略的 prefix，但部署时访问的是自己生成的 prefix。一旦早期 token 出错，后续状态分布会偏离训练数据，这就是 autoregressive exposure bias / distribution mismatch。

GKD 的解决思路是：让 Student 自己 rollout，再让 Teacher 在 Student 实际访问的序列上提供 token-level feedback；同时用  $\lambda$  控制 fixed data 与 on-policy student data 的混合比例，并允许选择不同 divergence [6]。

| 方案            | 数据分布                                    | 优点                    | 风险                                   |
|---------------|-----------------------------------------|-----------------------|--------------------------------------|
| Offline SeqKD | Teacher trajectories                    | 简单稳定                  | Student state coverage 差             |
| Supervised KD | 固定 dataset + teacher logits             | dense token feedback  | 仍有 train/inference mismatch          |
| On-policy KD  | Student trajectories + teacher feedback | 直接修复 Student mistakes | teacher serving 成本；早期 Student 太差时不稳定 |
| Mixed GKD     | fixed + student trajectories            | 兼顾稳定与覆盖               | 需要调 $\lambda$ 与 divergence           |

### 2026 前沿

Entropy-Aware On-Policy Distillation 指出，纯 reverse KL 在 Teacher 高熵 token 上可能过度 mode-seeking、降低 diversity，因此在高熵区域加入 forward KL。其报告在六个数学推理 benchmark 上，相比基线 on-policy distillation 提升多个 Qwen3 Student 的 Pass@8 [14]。

## 9. Teacher 选择：Teacher 越强 $\neq$ Student 越强

蒸馏是一个带宽受限的信息传输问题。Student 容量不足时，Teacher 的复杂分布、长 CoT、多模态/多策略输出可能超出其可吸收范围。容量差距研究长期观察到“更大的 Teacher 不必然教出更好的 Student”，并尝试建立 teacher/student scale 的 capacity-gap 规律 [9]。

| Teacher 评估维度 | 只看 Teacher benchmark 的问题 | 更好的选择标准                                   |
|--------------|--------------------------|-------------------------------------------|
| 正确率          | 忽略 Student 是否学得会         | Student distillation gain                 |
| CoT 长度       | 长不等于高信息密度                | 单位 token 的有效 reasoning signal             |
| 输出复杂度        | 可能超 Student 表达能力         | 与 Student capacity/architecture 匹配        |
| 多样性          | 过低会模式坍塌；过高会难学            | 正确且互补的策略覆盖                                |
| 校准/熵         | 高熵处 token target 更难      | 按 teacher uncertainty 动态选择 divergence     |
| 推理成本         | 只看质量会导致 distill 成本失控     | accepted useful signal / teacher GPU-hour |

2025 一项对三个 reasoning Teacher、1.89M shared queries 的并行蒸馏研究也发现：不同 Teacher 的“正确答案”作为蒸馏来源产生的 Student 表现并不相同；trace length diversity 与 perplexity 等属性可能影响 Student [13]。

## 10. Student 设计：容量、架构与初始化决定“可蒸馏边界”

| Student 变量            | 影响                                      | 工程建议                                                  |
|-----------------------|-----------------------------------------|-------------------------------------------------------|
| 参数规模                  | 决定可表达的 Teacher mode/策略复杂度               | 不要只选最小模型；先做 teacher-student gap sweep                 |
| Base vs Instruct init | 决定是否已有基本行为先验                            | Reasoning distill 常从能力合适的 base/instruct checkpoint 起步 |
| Tokenizer             | 跨 tokenizer 时 token-level logits 不可直接对齐 | 优先 sequence-level 或做 vocabulary mapping               |
| Architecture          | hidden-state 对齐难度不同                     | black-box sequence KD 对跨架构更稳健                         |
| Context window        | Teacher 长 CoT 可能超 Student context       | trace truncation/segmentation/summary                 |
| MoE / Dense           | 参数与激活结构不同                               | 可先结构转换，再通过 KD 恢复性能                                    |

## 11. Distillation Loss Design：不要把 KL 成固定答案

| Loss / Signal                 | Know-Why                           | Know-How                              |
|-------------------------------|------------------------------------|---------------------------------------|
| Hard SFT CE                   | 行为最直接                              | 适合 black-box / verified sequence      |
| Forward KL                    | 覆盖 Teacher 分布                      | 适合需要 mode coverage 的 token KD         |
| Reverse KL                    | 压制 Student 在 Teacher 不支持区域的概率      | 适合 generative precision，但注意 diversity |
| JSD / generalized divergence  | 在 mode-covering / mode-seeking 间折中 | GKD 类方法可任务化选择                         |
| Rationale multitask           | 把中间知识显式监督                          | label loss + rationale loss 加权        |
| Feature / hidden matching     | 迁移内部 representation                | 需要 white-box + architecture mapping   |
| Preference / critique distill | Teacher 只给相对判断或纠错                  | 可接 DPO / correction SFT               |



### 推荐实验顺序

先做 ① Verified sequence SFT baseline；再做 ② + token KD；再做 ③ on-policy KD；最后才引入 feature matching、复杂 divergence 或多阶段蒸馏。这样更容易识别真实增益来自“数据”还是“损失”。

## 12. Distillation Data Factory：蒸馏数据不是普通 Synthetic Data

Distillation Data 的价值取决于 Teacher-specific information 是否会被 Student 吸收。仅仅“大模型生成的数据”不自动等于 distillation data；如果 Teacher 输出与普通标注没有额外信息，Student 获得的只是另一份 SFT 数据。

| 数据层              | 核心字段                                          | 质量门槛                      |
|------------------|-----------------------------------------------|---------------------------|
| Prompt           | domain, difficulty, source, target capability | 覆盖 Student capability gap |
| Teacher Response | answer, rationale, tool calls                 | 正确、非模板化                   |
| Teacher Metadata | model, decoding, temperature, version         | 可追踪                       |
| Verifier         | score, test result, judge                     | 独立于 Generator             |
| Student Probe    | base pass/fail, perplexity, policy likelihood | 确认“可学且有缺口”                |
| Selection        | novelty, diversity, cost                      | 避免重复复制 Teacher style      |

## 13. Distillation Scaling Laws：什么时候“蒸馏”值得？

ICML 2025 的 Distillation Scaling Laws 研究把 Teacher 与 Student compute allocation 纳入统一模型。其结论之一是：当 Teacher 已存在，或同一 Teacher 要服务多个 Student 时，distillation 在一定 compute 区间内可以优于 supervised learning；但如果只训练一个 Student 且 Teacher 也必须从头训练，直接 supervised training 往往更合适 [8]。

| 场景                         | 更可能优先选择                                  | Know-Why                           |
|----------------------------|------------------------------------------|------------------------------------|
| Teacher 已经存在               | Distillation                             | Teacher training cost 已 sunk       |
| 同一 Teacher 蒸馏多个 Student    | Distillation                             | Teacher 成本可在多个 Student 间摊销         |
| 只要一个 Student，Teacher 也要新训练 | Direct supervised / pretraining          | 为蒸馏先训练 Teacher 可能不 compute-optimal |
| 需要低延迟部署多个尺寸                | Distillation family                      | 一次能力发现，多次能力压缩                      |
| Teacher API 极贵             | Verified sequence + aggressive filtering | 减少 teacher token 消耗                |

### 工程 KPI

不要只看 Student accuracy。更好的目标是：Quality Retention / Total Distillation Cost，以及每节省 1 ms latency、1 GB memory、1 美元 inference cost 需要支付多少训练成本。

## 14. Architecture Distillation：从 MoE 到 Dense

蒸馏不仅是“知识压缩”，也可以是架构迁移。2026 的 MoE-to-Dense 工作在 Qwen3-30B-A3B 等模型上系统测试 expert scoring/grouping，再用约 4B-token distillation 恢复 dense model 性能；其控制实验报告 matched parameter count 下，MoE-to-dense 相比 dense-to-dense pruning 平均下游准确率高 6.3 个百分点，并有更快训练 wall-clock [15]。

| 阶段                   | MoE→Dense 典型逻辑              |
|----------------------|-----------------------------|
| Expert scoring       | 判断哪些专家对目标能力最重要              |
| Selection / Grouping | 把稀疏专家压成有限 dense FFN 结构      |
| Initialization       | 通过权重组合获得 Student 初始点        |
| Distillation         | Teacher 纠正结构压缩造成的行为损失       |
| Evaluation           | 同时看质量、dense inference、显存、吞吐 |

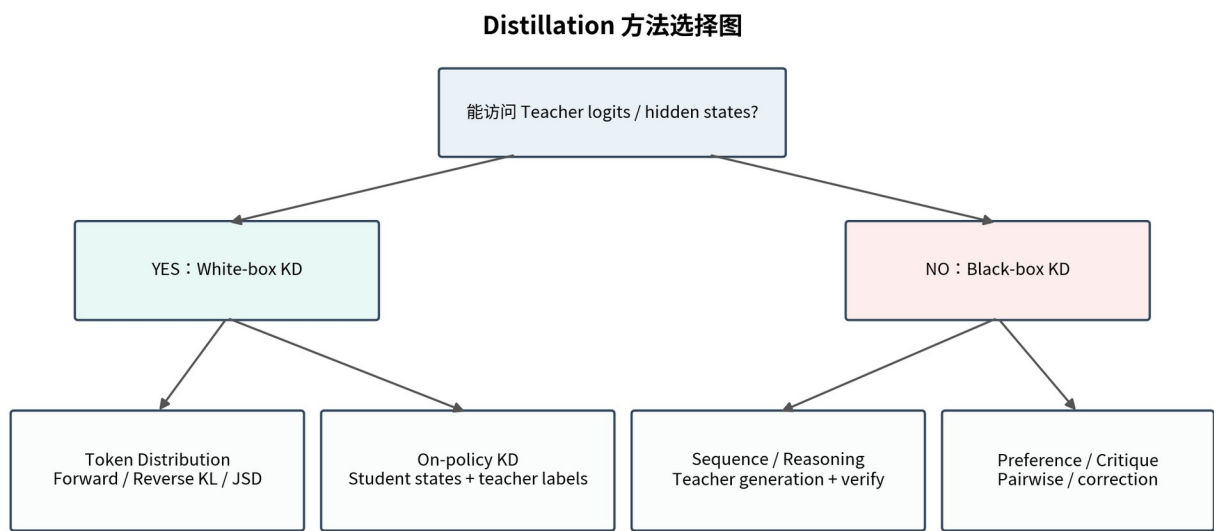
## 15. Self-Distillation：Teacher 和 Student 可以是“同一个模型”吗？

可以。Self-distillation 并不要求 Teacher 参数量更大，它可以来自同一模型的更强解码策略、ensemble、长 CoT、搜索后轨迹或显式 reasoning branch。关键是 Teacher channel 必须比 Student training target 含有更多有效信息。

| Self-distillation 来源            | Teacher 为什么更强                  | Student 学什么                    |
|---------------------------------|--------------------------------|--------------------------------|
| Best-of-N self-generation       | 搜索提高了 output quality           | 高质量序列                          |
| Long CoT → short/implicit CoT   | Teacher 使用更多 test-time compute | 压缩后的 reasoning representation  |
| Critique → corrected answer     | Teacher 多一次反思/纠错               | error recovery 行为              |
| Old checkpoint ensemble         | 多个模型平均降低方差                     | soft targets                   |
| Explicit → continuous reasoning | 显式 CoT 提供监督                    | 内部 latent reasoning；如 CODI 类方向 |

这类方法把蒸馏从“模型压缩”扩展成“计算压缩”：Teacher 可以用更长推理、更大 search budget，Student 学习后以更少 token 或更小模型重现部分效果。

## 16. Distillation 与 SFT / DPO / RL 的接口



工程上经常混合：sequence data 负责“教会行为”，token-level KD 负责“细化分布”，on-policy 负责“修复 student 自身错误状态”。

图 4：根据 Teacher 可访问性选择 Distillation 形态

| 阶段组合                         | 典型目的                                             | 注意事项                              |
|------------------------------|--------------------------------------------------|-----------------------------------|
| Teacher → Student SFT        | 行为/推理复制                                          | 最简单，但只保留 sampled sequence         |
| SFT + Logit KD               | 行为 + token distribution                          | 需要 white-box teacher              |
| Distill → DPO                | 先复制能力，再对齐偏好                                      | preference data 应 near-policy     |
| RL Teacher → Distill Student | 搜索→压缩                                            | Student 不必自己重复昂贵 RL               |
| Distill → Student RL         | 先拿到高质量先验，再探索边界                                   | 适合 Student 仍有可验证任务与预算             |
| On-policy KD + RL            | dense teacher signal + sparse environment reward | 系统复杂，但可同时修 behavior 与 task reward |

## 17. Production Distillation Pipeline

1. 定义 Deployment Contract：目标参数量、延迟、显存、吞吐、上下文长度和质量保真阈值。
2. 建立 Student Baseline：先测 base/instruct student 的能力缺口，而不是先生成海量 Teacher data。
3. 选择 Teacher：用 “Student distillation gain” 而非 Teacher benchmark 单独排名。
4. 设计 Prompt Distribution：覆盖真实需求 + capability gaps + learnable frontier。
5. 生成 Teacher candidates：记录 decoding config 与 Teacher version。
6. 独立验证：math/code 优先 deterministic verifier；开放任务用 judge ensemble / human audit。
7. 过滤与压缩 trace：去错误、近重复、无效长 CoT；保持 reasoning strategy diversity。
8. 训练 baseline：先 sequence SFT；再逐步加入 token KD / on-policy KD。
9. 行为评测：不仅看 task accuracy，还看 length、style、calibration、hallucination、tool correctness。
10. 部署评测：测真实 latency、VRAM、throughput、token cost。

11. Failure mining: Student 失败样本进入下一轮 targeted distillation。

## 18. Production Data / Training Contract 示例

```
distillation:
  objective:
    target_student: qwen-7b
    latency_p95_ms: 180
    quality_retention: ">= 0.92 teacher"
  teacher:
    model: reasoning-teacher-v3
    access: black_box
    decoding:
      n: 8
      temperature: 0.7
  data:
    prompt_mix:
      math: 0.35
      code: 0.35
      general_reasoning: 0.20
      tool_use: 0.10
    verifier_required: true
    dedup: semantic
  training:
    stage_1: verified_sequence_sft
    stage_2: optional_on_policy_kd
    divergence: reverse_kl
  eval:
    capability: [aime, math500, livecodebench]
    deployment: [latency, vram, throughput]
    behavior: [format, length, calibration]
```

## 19. Distillation Evaluation：如何证明 Student 真正“继承”了 Teacher？

| Eval 层               | 指标                                      | 需要回答的问题                                     |
|----------------------|-----------------------------------------|---------------------------------------------|
| Task capability      | accuracy, pass@1, pass@k                | 核心能力保留多少？                                   |
| Teacher agreement    | KL, pairwise agreement, behavior match  | 是否只是 benchmark 巧合？                          |
| Generalization       | OOD / hard split                        | 是否只记住 Teacher data？                         |
| Calibration          | ECE / NLL / confidence                  | Student 是否更自信地犯错？                           |
| Diversity            | entropy, distinct-n, pass@k             | reverse KL / single-seq 是否导致 mode collapse？ |
| Reasoning efficiency | tokens-to-correct, response length      | 是否只是复制冗长 CoT？                               |
| Deployment           | latency, VRAM, throughput, \$/1M tokens | 压缩价值是否成立？                                   |
| Safety/behavior      | regression slices                       | 能力蒸馏是否破坏已有行为约束？                             |

### 最重要的 Ablation

至少比较：Student baseline、Teacher-sequence SFT、+Verifier filtering、+multi-sample diversity、+token KD、+on-policy KD。否则无法判断提升来自“Teacher 更好”“数据更多”还是“Distillation objective 更好”。

20. 18 类常见失败模式与修复

| Failure                     | 表现                         | 修复                                       |
|-----------------------------|----------------------------|------------------------------------------|
| 1. Teacher worship          | 只看 teacher benchmark       | 用 student gain 反向选择 teacher              |
| 2. Wrong-trace copying      | Teacher CoT 有错仍被训练         | Verifier + rejection sampling            |
| 3. Capacity mismatch        | Student 学不动复杂 trace        | 简化/分段 trace；缩小 teacher gap               |
| 4. Exposure bias            | 只用 teacher trajectories    | 加入 on-policy student rollout             |
| 5. Mode collapse            | single-seq / reverse KL 过强 | multi-sample；熵监控；混合 FKL/JSD              |
| 6. Style imitation          | 学会语气但没学能力                  | 独立 correctness eval / hidden tests       |
| 7. Length imitation         | Student 复制 Teacher 冗长 CoT  | length-normalized selection              |
| 8. Dataset monoculture      | 所有数据来自单一 Teacher           | multi-teacher / real-data anchor         |
| 9. Verifier leakage         | 训练集围绕 verifier 漏洞          | hidden verifier / adversarial audit      |
| 10. Tokenizer mismatch      | 无法做 token KL               | sequence KD 或 vocab mapping              |
| 11. Off-policy staleness    | 固定 data 与 Student 已偏离      | 周期性 regenerate / on-policy               |
| 12. Over-distillation       | 训练过久导致 general ability 回退  | early stop + regression suite            |
| 13. Benchmark contamination | Teacher 生成含测试答案            | dedup + provenance + contamination audit |
| 14. Weak teacher diversity  | 只产生一个解法                    | N-sampling + strategy diversity          |
| 15. Teacher cost explosion  | 生成 N×long-CoT 太贵           | adaptive N / early verifier / cache      |
| 16. Eval mismatch           | 只看 academic benchmark      | 加入真实 deployment eval                     |
| 17. No compute accounting   | 忽略 Teacher 训练/推理成本         | 总成本 + amortization 模型                    |
| 18. Distill everything      | 所有能力都压到最小 Student          | 按 product SLA 决定保留/放弃能力                  |

21. 方法选择矩阵：什么时候用哪种 Distillation？

| 场景                   | 优先方案                                  | 原因                               |
|----------------------|---------------------------------------|----------------------------------|
| 只有 API Teacher       | Verified Sequence / Reasoning SFT     | black-box 最现实                    |
| 可访问 logits           | SFT + token KD                        | 利用 dense distribution signal     |
| Student 自回归错误累积      | On-policy GKD                         | 覆盖 Student 自己状态                  |
| 数学/代码 reasoning      | Search → Verify → Distill             | 可验证轨迹最有价值                        |
| Teacher/Student 差距巨大 | trace simplify + intermediate teacher | 减少 capacity mismatch             |
| 要求多样输出               | multi-sample + FKL/JSD mix            | 避免纯 mode-seeking collapse        |
| 极低延迟                 | smaller dense + distill / MoE→Dense   | 将 Teacher 能力压入部署结构               |
| Teacher 很贵           | 高价值 prompt + verifier + cache         | 优化 useful signal / teacher token |

## 22. Distillation Factory 成熟度模型

| 等级                    | 特征                                                                      | 关键升级                                  |
|-----------------------|-------------------------------------------------------------------------|---------------------------------------|
| L0 - Copy             | 拿 Teacher answers 做 SFT                                                 | 先做 verifier / provenance              |
| L1 - Curated          | Teacher data + filtering + eval                                         | 数据质量可控                                |
| L2 - Multi-signal     | sequence + logits/rationale/preference                                  | 监督粒度多样化                               |
| L3 - On-policy        | Student rollout + Teacher feedback                                      | 修复 exposure bias / stale data         |
| L4 - Adaptive Factory | gap mining → adaptive generation → verify → train → deployment feedback | 按 Student 与成本动态调 Teacher、N、trace、loss |

## 23. Know-Why / Know-How 总结

| Know-Why                                         | Know-How                                                                                |
|--------------------------------------------------|-----------------------------------------------------------------------------------------|
| 蒸馏的价值来自 “Teacher 已经支付过的搜索/训练成本” 可以被多个 Student 摊销 | 把 Teacher 训练成本、生成成本、Student 训练成本一起计入 ROI                                                |
| Student 容量有限，因此不是 Teacher 的全部知识都可迁移              | 先 probe Student，再决定 trace granularity、teacher scale 与 divergence                        |
| 固定 Teacher data 不能覆盖 Student 部署时的状态分布            | 周期性加入 on-policy student rollouts                                                        |
| Reasoning trace 的 correctness 比 “像 Teacher” 更重要  | math/code 优先 deterministic verifier；开放任务做独立 judge audit                                 |
| 更强 Teacher 不一定更适合某个 Student                      | 以 distilled Student 结果做 teacher selection                                               |
| 蒸馏是一个系统工程，而不是单一 loss                             | Data Factory + Teacher Serving + Verifier + Training + Eval + Deployment telemetry 共同优化 |

最终心智模型

Distillation = “能力发现” 与 “能力部署” 之间的编译器。Teacher/RL/Search 负责发现高质量策略；Distillation 负责在 Student 的容量、延迟和成本约束下，把最值得保留的策略重新编码。

## 24. 核心论文与推荐阅读顺序

[1] Hinton, Vinyals, Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.

[2] Kim, Rush. Sequence-Level Knowledge Distillation. EMNLP 2016 / arXiv:1606.07947.

[3] Hsieh et al. Distilling Step-by-Step! Findings of ACL 2023 / arXiv:2305.02301.

[4] Mukherjee et al. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707, 2023.

[5] Gu et al. MiniLLM: On-Policy Distillation of Large Language Models. ICLR 2024 / arXiv:2306.08543.

[6] Agarwal et al. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes (GKD). ICLR 2024 / arXiv:2306.13649.

- [7] DeepSeek-AI et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, revised 2026.
- [8] Busbridge et al. Distillation Scaling Laws. ICML 2025 / arXiv:2502.08606.
- [9] Zhang et al. Towards the Law of Capacity Gap in Distilling Language Models. arXiv:2311.07052.
- [10] Chen et al. Unveiling the Key Factors for Distilling Chain-of-Thought Reasoning. arXiv:2502.18001, 2025.
- [11] Luo et al. Deconstructing Long Chain-of-Thought: A Structured Reasoning Optimization Framework for Long CoT Distillation. arXiv:2503.16385, 2025.
- [12] Zhao et al. 1.4 Million Open-Source Distilled Reasoning Dataset to Empower Large Language Model Training. arXiv:2503.19633, 2025.
- [13] Tian et al. Not All Correct Answers Are Equal: Why Your Distillation Source Matters. arXiv:2505.14464, 2025.
- [14] Jin et al. Entropy-Aware On-Policy Distillation of Language Models. ICLR 2026 / arXiv:2603.07079.
- [15] Kim et al. Pruning and Distilling Mixture-of-Experts into Dense Language Models. arXiv:2605.28207, 2026.

## 推荐阅读路线

12. 先读 [1][2]: 建立 logit KD 与 sequence KD 的经典框架。
13. 再读 [3][4]: 理解 rationale / explanation trace 为什么把蒸馏从“压缩分类器”带到“复制推理行为”。
14. 读 [5][6]: 理解 generative LLM 中 reverse KL、exposure bias 与 on-policy distillation。
15. 读 [7]: 理解现代 reasoning post-training 中 Search/RL → Distill 的工程范式。
16. 读 [8][9]: 理解 compute allocation 与 teacher-student capacity gap。
17. 最后读 [10]-[15]: 进入 2025-2026 的 long-CoT、teacher-source、entropy-aware、MoE→Dense 等前沿问题。

— End of Handbook —